Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого

ОТЧЁТ

по лабораторной работе № 3

«Проектирование схемы базы данных по описанию задачи с помощью LLM»

Вариант 17 «Театральная касса»

по дисциплине «Автоматизация научных исследований»

Выполнил студент группы 5040102/60201
Поляков В.

Санкт-Петербург
2026

1 ЗАДАНИЕ

Выбрано задание № 17 «Театральная касса». Формулировка использована без изменений:

Театральная касса

Есть список театров. Для каждого театра есть репертуар. Один и тот же спектакль может быть поставлен в разных театрах. Места в зрительном зале разделены минимум на 2 категории (по цене); есть информация об общем количестве мест каждой категории. На каждую постановку в каждом театре есть цены на билет (с учетом категорий мест). При покупке билетов на две и более постановок, человек получает скидку 10%, начиная со второй постановки. При покупке более 10 билетов на одну постановку, также есть скидка на все эти билеты 10%. Скидки суммируются.
Требуется:
- Поиск театра и периода постановки по названию;
- Поддержка продажи билетов на одну и несколько постановок;
- Поддержка возврата билетов с удержанием 50% заплаченного, если возврат происходит накануне спектакля (при частичном возврате билетов после их «оптовой» покупки в случае необходимости (если купленных билетов осталось <10) должен быть произведен перерасчет);
Отчеты:
- Репертуар театров города;
- Текущее состояние по проданным билетам на постановки выбранного театра;
- Финансовый отчет за квартал/год по всем театрам;
- Коэффициент «пустых» продаж (отношение количества возвращенных билетов к проданным);
- Коэффициент «успешности» (отношение количества реально проданных билетом к общему количеству мест (с учетом категорий мест)) по каждому театру и по каждой постановке за выбранный период времени.

Работа выполнена индивидуально; для сравнения схема получена от двух моделей.

2 МОДЕЛИ И ИСХОДНЫЙ ЗАПРОС

Таблица 1 — Использованные модели

Модель	**Доступ**	**Условия**
Qwen	бесплатная веб-версия, модель по умолчанию	новый диалог; запрос с замечаниями — в том же диалоге
Claude (claude-opus-5-5)	приложение Claude	ответ получен не в новом диалоге: до запроса в диалоге обсуждались задание, требования методических указаний и ответ Qwen; запрос с замечаниями — в том же диалоге

Обеим моделям отправлен одинаковый запрос по шаблону из методических указаний:

```
По приведённому ниже заданию спроектируй реляционную схему базы данных.
Выдай полный код DBML для вставки в https://dbdiagram.io/.
Нужна только схема: таблицы, поля, типы данных, ключи и связи.
Все операции и отчёты из условия должны быть реализуемы на её основе.
SQL-запросы и код приложения не нужны.
Если необходимы допущения, явно отметь их в комментариях DBML.

Полное условие задания:
[текст задания из раздела 1]
```

Полные ответы моделей сохранены в файлах *qwen_p0.txt* и *claude_p0.txt*, код DBML приведён в приложении А.

3 ИСХОДНЫЕ СХЕМЫ

Код обеих схем проверен разборщиком DBML *@dbml/core* 12.0.0 — тем же, что используется в dbdiagram.io. Ошибок синтаксиса не обнаружено, дополнительных обращений к моделям для исправления DBML не потребовалось. Диаграммы приведены на рисунках 1 и 2.

Qwen предложил 11 таблиц: города, театры, залы (*TheaterHall*), категории мест, спектакли (*Play*), постановки (*Production*) с периодом показа, показы (*Performance*), цены, заказы, позиции заказа (*OrderItem*) и билеты (*Ticket*). Модель самостоятельно добавила залы, которых нет в условии.

Claude предложил 10 таблиц: города, театры, категории мест (на уровне театра — допущение об одном зале), спектакли, постановки, цены, показы, покупки, билеты с раздельными скидками и таблицу возвратов *ticket_returns*.

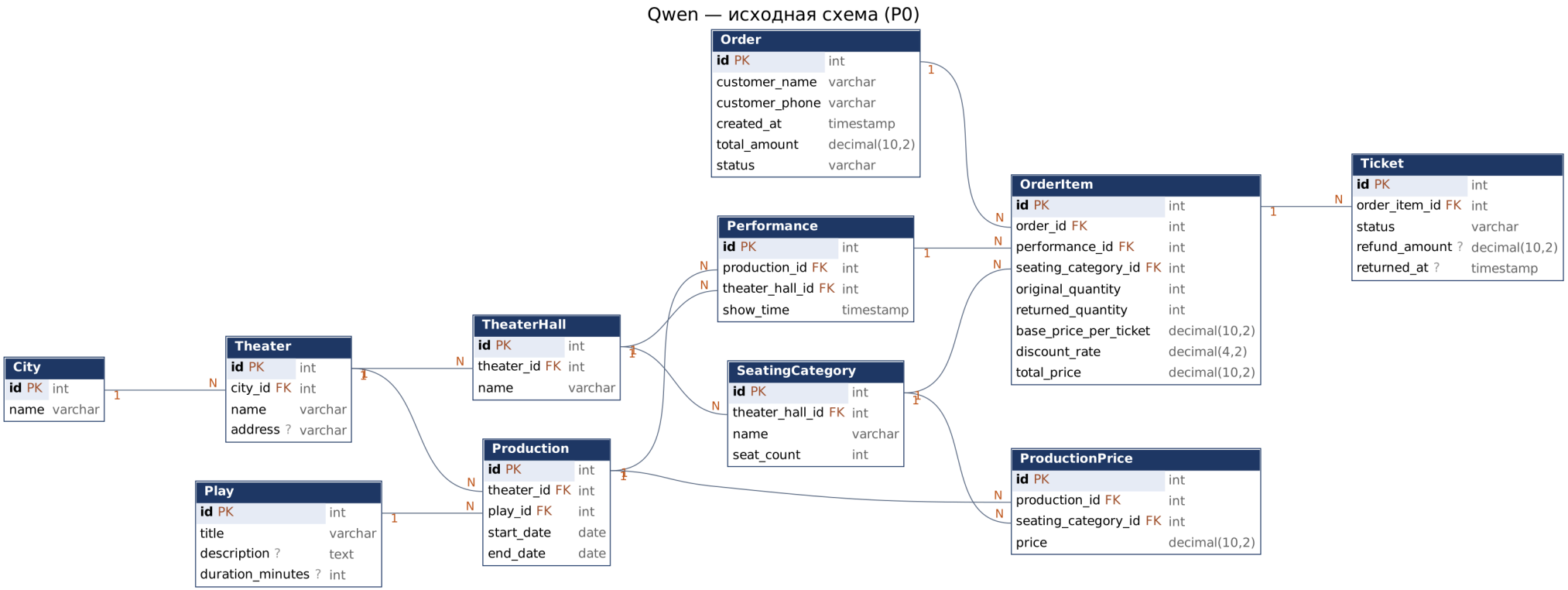


Рисунок 1 — Исходная схема Qwen (P0)

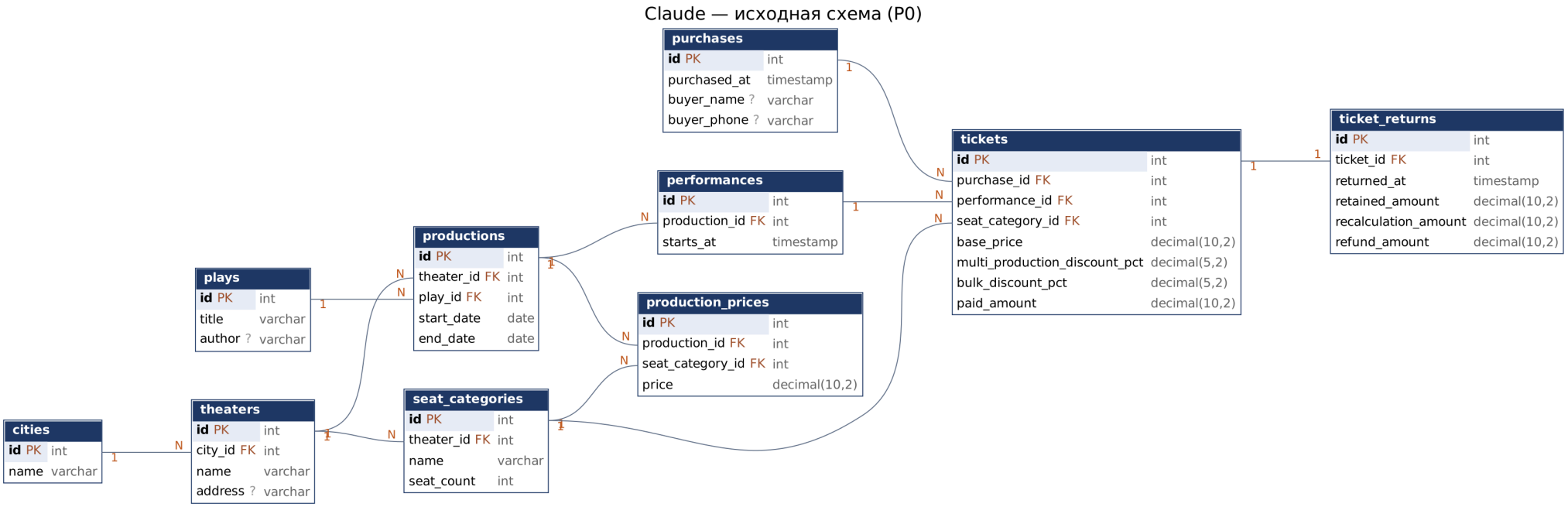


Рисунок 2 — Исходная схема Claude (P0)

4 КАЧЕСТВЕННЫЙ РАЗБОР ИСХОДНЫХ СХЕМ

Схемы сопоставлены с условием: проверено, хватает ли данных для операций и отчётов, различаются ли объекты, верно ли представлены связи, не дублируются ли факты и сохраняется ли история после возвратов.

Таблица 2 — Проблемы схемы Qwen (P0)

Проблема и место в схеме	**Жизненная ситуация**	**Последствие**	**Что следует изменить**
1. *OrderItem.discount_rate* — одно число, в котором суммированы обе скидки	В заказе 12 билетов на вторую постановку (discount_rate = 0,20), затем 3 билета возвращают	При значении 0,10 нельзя понять, какая скидка применена; перерасчёт при возврате неоднозначен	Хранить скидку за вторую постановку и оптовую скидку отдельно
2. Оптовая скидка считается по *OrderItem* (показ + категория), а по условию — по постановке	6 билетов в партер и 6 на балкон на одну постановку — две позиции заказа	Ни в одной позиции нет 10 билетов, порог «более 10» не определить	Связать скидку с постановкой в рамках заказа
3. Факт возврата хранится в *Ticket.status*, *OrderItem.returned_quantity*, *Order.status*; сумма — в *Order.total_amount* и *OrderItem.total_price*	При возврате обновили Ticket.status, но не returned_quantity	Отчёт о продажах и коэффициент «пустых» продаж дают разные числа	Хранить факт возврата в одном месте
4. Изменение цены оставшихся билетов при перерасчёте нигде не фиксируется	Куплено 12 билетов со скидкой, возвращено 3 (скидка отменена), затем возвращают ещё 1	Второй возврат считается от цены со скидкой — покупателю возвращают больше, чем он заплатил	Хранить сумму, заплаченную за каждый билет, и корректировки цены; отдельно — удержание 50%
5. Категория мест в *ProductionPrice* и *OrderItem* может быть из любого зала, зал показа — из любого театра	Постановка идёт в Большом зале, а билет продан в категорию Малого зала	Продан билет на несуществующие места; коэффициент «успешности» считается по чужой вместимости	Обеспечить ключами принадлежность зала театру постановки, категории — залу показа

6. Нет уникальности (production_id, seating_category_id) и (theater_hall_id, show_time)	Для одной категории введены две цены; в одном зале в одно время два показа	Неясно, по какой цене продавать; одни и те же места продаются дважды	Добавить уникальные индексы
7. Вместимость хранится только текущая (*SeatingCategory.seat_count*)	После реконструкции изменили число мест, нужен коэффициент за прошлый год	Прошлые показы считаются с новой вместимостью	Хранить вместимость с периодом действия

Таблица 3 — Проблемы схемы Claude (P0)

Проблема и место в схеме	**Жизненная ситуация**	**Последствие**	**Что следует изменить**
1. Категории мест привязаны к театру; предполагается один зал	В театре основная и камерная сцены с разным числом мест	Вместимость показа не определить, коэффициент «успешности» считается по всему театру; на камерную сцену можно продать билет на балкон	Ввести залы, привязать к ним категории и показы
2. *production_prices.seat_category_id* и *tickets.seat_category_id* могут указывать на категорию другого театра	При вводе цены или продаже выбрана категория «Партер» другого театра	Продан билет на места, которых нет в зале; отчёты по театру искажаются	Обеспечить ключами соответствие категории театру постановки и залу показа
3. Перерасчёт записывается только в *ticket_returns.recalculation_amount* возвращённого билета	Куплено 12 билетов со скидкой, возвращено 3, затем ещё 1	Второй возврат считается от цены со скидкой — покупатель получает лишнее	Хранить корректировки цены каждого оставшегося билета со ссылкой на вызвавший их возврат
4. Уникален только (production_id, starts_at), а не показ в зале	В одном зале на 19:00 назначены показы двух постановок	Одни и те же места продаются дважды	Уникальность (зал, время)

5. Вместимость хранится только текущая (*seat_categories.seat_count*)	После реконструкции изменили число мест, нужен отчёт за прошлый год	Прошлые показы считаются с новой вместимостью	Хранить вместимость с датой начала действия

Общие для обеих схем недостатки, которые реализуются только на уровне приложения: схема не запрещает продать больше билетов, чем мест в категории, и назначить показ вне периода постановки. Они не передавались моделям, так как не выражаются средствами реляционной схемы без триггеров.

5 ИСПРАВЛЕНИЕ СХЕМ

Списки проблем из таблиц 2 и 3 отправлены моделям в тех же диалогах по шаблону из методических указаний (полный текст — в приложении Б). Обе модели вернули полный DBML; ошибок синтаксиса нет. Код приведён в приложении А, диаграммы — на рисунках 3 и 4.

Qwen добавил таблицу *OrderProduction* (постановка в рамках заказа) с раздельными ставками скидок и суммарным количеством билетов, таблицу *SeatingCategoryCapacity* с периодом действия вместимости, удалил *returned_quantity* и *Order.status*, добавил в *Ticket* поля удержаний.

Claude добавил залы (*halls*), таблицу вместимости с датой начала действия, таблицу корректировок цены *price_adjustments*, уникальный индекс показа по залу и времени и составные внешние ключи, связывающие категорию, зал и постановку с одним театром.

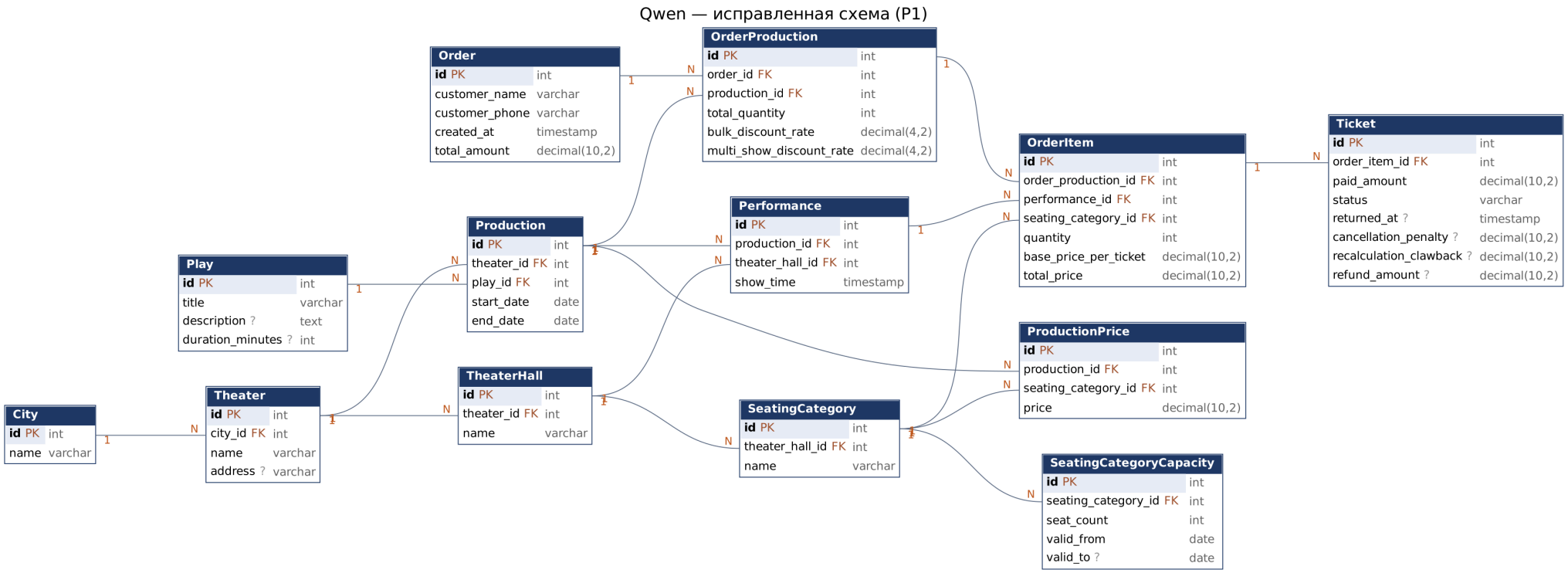


Рисунок 3 — Исправленная схема Qwen (P1)

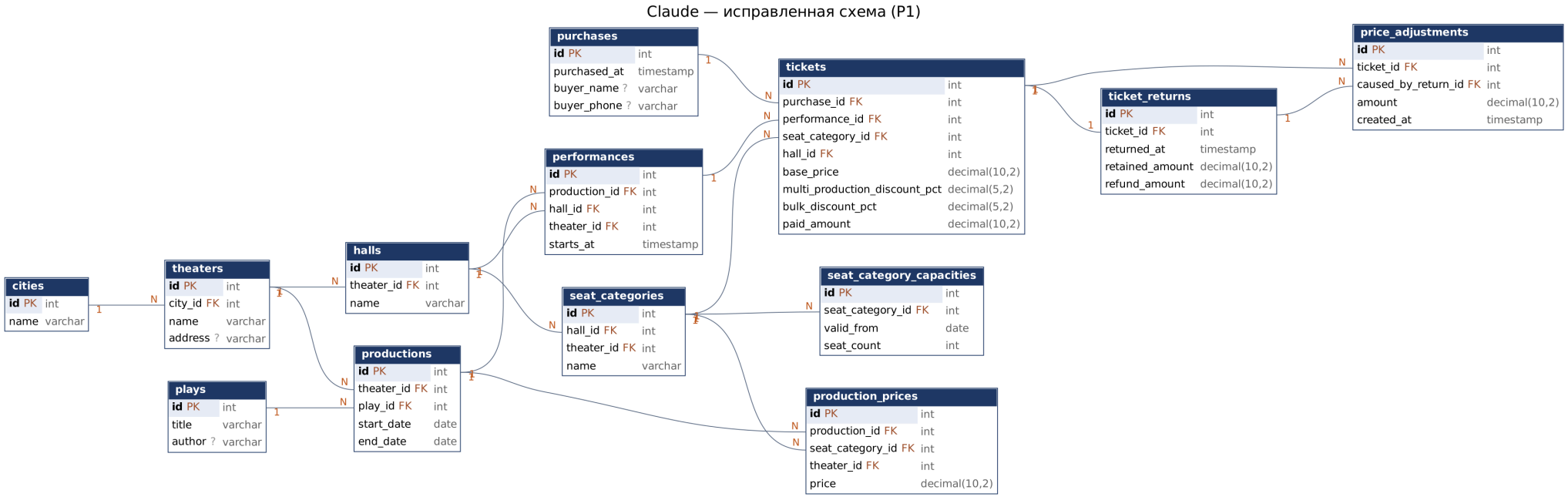


Рисунок 4 — Исправленная схема Claude (P1)

6 ПРОВЕРКА ИСПРАВЛЕННЫХ СХЕМ

6.1 Qwen

Таблица 4 — Обработка замечаний моделью Qwen

Замечание	**Результат**	**Пояснение**
1. Скидки в одном поле	исправлено	В *OrderProduction* хранятся *bulk_discount_rate* и *multi_show_discount_rate*
2. Оптовая скидка по позиции	исправлено	Новая таблица *OrderProduction* группирует билеты постановки в заказе
3. Дублирование возврата	частично	Удалены *returned_quantity* и *Order.status*, но остались *Order.total_amount*, *OrderItem.total_price*, а в *Ticket* возврат по-прежнему виден и по *status*, и по *returned_at*
4. Перерасчёт цены оставшихся билетов	**не исправлено**	Удержание *recalculation_clawback* записывается в возвращённый билет; *paid_amount* оставшихся билетов не меняется, поэтому повторный возврат снова считается от цены со скидкой
5. Категория из чужого зала	**не исправлено**	Ограничения добавлены только комментариями «BUSINESS LOGIC CONSTRAINT» — проверку перекладывают на приложение, ключи по-прежнему допускают ошибку
6. Уникальность	**не исправлено**	Уникальные индексы описаны только в комментариях, в DBML блока *indexes* нет — dbdiagram.io и сгенерированный SQL их не создают
7. История вместимости	исправлено	Таблица *SeatingCategoryCapacity* с *valid_from*, *valid_to*

Таблица 5 — Новые проблемы схемы Qwen (P1)

Проблема	**Жизненная ситуация**	**Последствие**
OrderProduction.total_quantity дублирует сумму *OrderItem.quantity*	Добавили билет в позицию, но не обновили total_quantity	Оптовая скидка применяется или отменяется неверно
OrderItem.performance_id не связан с *OrderProduction.production_id*	Позиция группы постановки А ссылается на показ постановки Б	Скидки и отчёты по постановкам считаются по неверной группе
Ставки скидок в *OrderProduction* перезаписываются при перерасчёте	После отмены оптовой скидки bulk_discount_rate меняют на 0	Теряется информация о том, по какой скидке были проданы уже возвращённые билеты

Qwen заявил, что «все 7 замечаний учтены», однако три из них исправлены только на словах: уникальность и принадлежность категории залу описаны в комментариях, но не выражены в DBML, а перерасчёт цены оставшихся билетов по-прежнему не фиксируется.

6.2 Claude

Таблица 6 — Обработка замечаний моделью Claude

Замечание	**Результат**	**Пояснение**
1. Один зал на театр	исправлено	Добавлена таблица *halls*; категории и показы привязаны к залу
2. Категория другого театра	исправлено	Составные внешние ключи: категория, зал и постановка — одного театра; категория билета — из зала показа
3. Перерасчёт	исправлено	Таблица *price_adjustments* со ссылкой на вызвавший возврат; поле *recalculation_amount* удалено
4. Два показа в зале	исправлено	Уникальный индекс (hall_id, starts_at)
5. История вместимости	исправлено	Таблица *seat_category_capacities* с *valid_from*

Таблица 7 — Оставшиеся спорные места схемы Claude (P1)

Место в схеме	**Ситуация**	**Последствие**
Поля *theater_id* и *hall_id* повторяются в нескольких таблицах	Нужны для составных внешних ключей; рассогласование невозможно — ключ его не пропустит	Схема сложнее для чтения
ticket_returns.refund_amount вычисляется из других полей	Хранится как факт выплаты	При ручной правке удержания сумма выплаты не пересчитается
Продажа сверх вместимости не запрещена схемой	Количество проданных билетов на категорию показа превысило вместимость	Проверка возможна только в приложении (общая проблема обеих схем)
Дата показа не проверяется по периоду постановки	Показ назначен после end_date	Проверка только в приложении (общая проблема обеих схем)

7 СРАВНЕНИЕ МОДЕЛЕЙ

Таблица 8 — Сравнение результатов

Показатель	**Qwen**	**Claude**
Ошибки синтаксиса DBML	нет (P0 и P1)	нет (P0 и P1)
Таблиц в P0 / P1	11 / 13	10 / 13
Найдено проблем в P0	7	5
Исправлено полностью / частично / нет	3 / 1 / 3	5 / 0 / 0
Новые проблемы в P1	3	0 (есть спорные решения)
Как оформлены ограничения	часть ограничений — только в комментариях	уникальные индексы и составные внешние ключи в DBML
Залы и категории	были в P0	добавлены в P1 по замечанию

Различия объясняются в первую очередь моделями, а не запросами: исходный запрос был одинаковым, запросы с замечаниями построены по одному шаблону. Однако Claude получил исходный запрос не в новом диалоге и видел требования к разбору и ответ Qwen, поэтому его исходная схема не является независимой — это ограничивает сравнение первых ответов. Различие в обработке замечаний от этого не зависит: Qwen переложил часть ограничений в комментарии и приложение, Claude выразил их ключами и индексами.

ВЫВОДЫ

1. Обе модели сразу выдали синтаксически корректный DBML; ошибок при построении диаграмм не было.
2. Обе исходные схемы покрывали основные сущности задания, но содержали ошибки, проявляющиеся только в жизненных ситуациях: при частичном возврате «оптовой» покупки, при наличии нескольких залов, при изменении вместимости, при ошибочной категории мест.
3. Самым сложным требованием задания оказался перерасчёт при частичном возврате: в исходных схемах обеих моделей изменение цены оставшихся билетов не сохранялось.
4. Qwen исправил 3 замечания из 7 полностью и 1 частично; три замечания модель отметила как учтённые, но выразила только комментариями. Кроме того, появились три новые проблемы, в том числе новое дублирование (*total_quantity*). Для выбранной задачи схема Qwen (P1) пригодна только при условии, что приложение реализует перечисленные проверки.
5. Claude исправил все 5 замечаний средствами схемы; плата за это — повторяющиеся поля *theater_id* и *hall_id* для составных ключей и более сложная структура. Схема Claude (P1) пригодна для задачи; ограничения вместимости и периода показа остаются задачей приложения.
6. Успешное построение диаграммы не означает правильности схемы, а утверждение модели «все замечания учтены» требует проверки: часть исправлений может оказаться только в комментариях.

ПРИЛОЖЕНИЕ А. КОД DBML

A.1 Qwen, исходная схема (P0)

```
//
================================================================================
// ДОПУЩЕНИЯ И ПОЯСНЕНИЯ К СХЕМЕ:
// 1. "Постановка" (Production) моделируется как абстрактный прокат
спектакля
//    в конкретном театре с периодом показа (start_date, end_date).
// 2. "Представление" (Performance) — это конкретный сеанс (дата и время)
//    в рамках Постановки. Билеты продаются именно на конкретное
Представление.
// 3. Добавлена сущность "Зал" (TheaterHall), так как в театре может быть
//    несколько залов, и общее количество мест по категориям привязано к
залу.
// 4. Для поддержки перерасчета при частичном возврате (если осталось < 10
билетов),
//    в OrderItem хранятся original_quantity и returned_quantity, а также
//    примененная ставка скидки (discount_rate). Это позволяет приложению
//    вычислить потерю скидки для оставшихся билетов и удержать эту сумму
//    из возврата (refund_amount) за возвращаемые билеты.
// 5. Возврат "накануне спектакля" с удержанием 50% реализуется на уровне
//    приложения путем сравнения Ticket.returned_at и
Performance.show_time.
//    Схема предоставляет для этого все необходимые поля.
// 6. Суммирование скидок (до 20%) отражается в поле discount_rate
(например, 0.20).
//
================================================================================

Table City {
  id int [pk, increment]
  name varchar [not null] // Название города
}

Table Theater {
  id int [pk, increment]
  city_id int [not null]
  name varchar [not null] // Название театра
  address varchar
}

Table TheaterHall {
  id int [pk, increment]
  theater_id int [not null]
  name varchar [not null] // Название зала (например, "Большой зал")
}

Table SeatingCategory {
  id int [pk, increment]
  theater_hall_id int [not null]
  name varchar [not null] // Название категории (например, "Партер",
"Балкон")
  seat_count int [not null] // Общее количество мест этой категории в зале
}
```

```
Table Play {
  id int [pk, increment]
  title varchar [not null] // Название спектакля
  description text
  duration_minutes int
}

Table Production {
  id int [pk, increment]
  theater_id int [not null]
  play_id int [not null]
  start_date date [not null] // Начало периода постановки
  end_date date [not null] // Конец периода постановки
}

Table Performance {
  id int [pk, increment]
  production_id int [not null]
  theater_hall_id int [not null] // Зал, в котором проходит конкретный
сеанс
  show_time timestamp [not null] // Дата и время начала представления
}

Table ProductionPrice {
  id int [pk, increment]
  production_id int [not null]
  seating_category_id int [not null]
  price decimal(10, 2) [not null] // Цена билета на данную постановку для
данной категории
}

Table "Order" {
  id int [pk, increment]
  customer_name varchar [not null]
  customer_phone varchar [not null]
  created_at timestamp [not null, default: `now()`]
  total_amount decimal(10, 2) [not null] // Итоговая сумма заказа с учетом
всех скидок
  status varchar [not null] // 'active', 'partially_returned',
'fully_returned'
}

Table OrderItem {
  id int [pk, increment]
  order_id int [not null]
  performance_id int [not null]
  seating_category_id int [not null]
  original_quantity int [not null] // Исходное количество билетов в позиции
(для проверки порога "10 билетов")
  returned_quantity int [not null, default: 0] // Количество уже
возвращенных билетов в этой позиции
  base_price_per_ticket decimal(10, 2) [not null] // Цена за билет без
скидок
  discount_rate decimal(4, 2) [not null, default: 0.0] // Суммарная ставка
примененной скидки (например, 0.10 или 0.20)
  total_price decimal(10, 2) [not null] // Итоговая стоимость позиции
}
```

```
Table Ticket {
  id int [pk, increment]
  order_item_id int [not null]
  status varchar [not null, default: 'sold'] // 'sold' или 'returned'
  refund_amount decimal(10, 2) // Фактическая сумма возврата по этому
билету (с учетом удержаний и перерасчетов)
  returned_at timestamp // Дата и время оформления возврата
}

//
==========================================================================
// СВЯЗИ (RELATIONSHIPS)
//
==========================================================================

Ref: Theater.city_id > City.id
Ref: TheaterHall.theater_id > Theater.id
Ref: SeatingCategory.theater_hall_id > TheaterHall.id

Ref: Production.theater_id > Theater.id
Ref: Production.play_id > Play.id

Ref: Performance.production_id > Production.id
Ref: Performance.theater_hall_id > TheaterHall.id

Ref: ProductionPrice.production_id > Production.id
Ref: ProductionPrice.seating_category_id > SeatingCategory.id

Ref: OrderItem.order_id > "Order".id
Ref: OrderItem.performance_id > Performance.id
Ref: OrderItem.seating_category_id > SeatingCategory.id

Ref: Ticket.order_item_id > OrderItem.id
```

A.2 Claude, исходная схема (P0)

```
// Театральная касса — реляционная схема
//
// ДОПУЩЕНИЯ:
// 1. В каждом театре один зрительный зал; категории мест и их количество
заданы для театра.
// 2. «Спектакль» (plays) — произведение, «постановка» (productions) —
спектакль в конкретном
//    театре с периодом показа, «показ» (performances) — конкретная дата и
время.
// 3. Билет продаётся на конкретный показ и категорию мест; конкретные
номера мест не хранятся,
//    свободные места = количество мест категории − непроданные/
невозвращённые билеты.
// 4. Покупка (purchases) — одна операция в кассе; скидки считаются внутри
одной покупки:
//    10% на билеты второй и следующих постановок, 10% на все билеты
постановки,
//    если на неё куплено больше 10 билетов. Скидки суммируются.
// 5. Возврат возможен не позднее дня накануне показа; накануне
удерживается 50% заплаченного.
// 6. При частичном возврате «оптовой» покупки, если на постановку осталось
```

```
меньше 10 билетов,
//    оптовая скидка с оставшихся билетов отменяется, разница удерживается
из суммы возврата.
// 7. Билет считается возвращённым, если для него есть запись в
ticket_returns.

Table cities {
  id int [pk, increment]
  name varchar [not null]
}

Table theaters {
  id int [pk, increment]
  city_id int [not null, ref: > cities.id]
  name varchar [not null, note: 'Названия театров могут совпадать']
  address varchar
}

Table seat_categories {
  id int [pk, increment]
  theater_id int [not null, ref: > theaters.id]
  name varchar [not null, note: 'Партер, амфитеатр, балкон...']
  seat_count int [not null, note: 'Общее количество мест этой категории']

  indexes {
    (theater_id, name) [unique]
  }
}

Table plays {
  id int [pk, increment]
  title varchar [not null]
  author varchar
}

Table productions {
  id int [pk, increment]
  theater_id int [not null, ref: > theaters.id]
  play_id int [not null, ref: > plays.id]
  start_date date [not null, note: 'Начало периода постановки']
  end_date date [not null, note: 'Окончание периода постановки']
}

Table production_prices {
  id int [pk, increment]
  production_id int [not null, ref: > productions.id]
  seat_category_id int [not null, ref: > seat_categories.id, note:
'Категория того же театра, что и постановка']
  price decimal(10,2) [not null]

  indexes {
    (production_id, seat_category_id) [unique]
  }
}

Table performances {
  id int [pk, increment]
  production_id int [not null, ref: > productions.id]
```

```
  starts_at timestamp [not null]

  indexes {
    (production_id, starts_at) [unique]
  }
}

Table purchases {
  id int [pk, increment]
  purchased_at timestamp [not null]
  buyer_name varchar
  buyer_phone varchar
}

Table tickets {
  id int [pk, increment]
  purchase_id int [not null, ref: > purchases.id]
  performance_id int [not null, ref: > performances.id]
  seat_category_id int [not null, ref: > seat_categories.id]
  base_price decimal(10,2) [not null, note: 'Цена по прайсу на момент
покупки']
  multi_production_discount_pct decimal(5,2) [not null, default: 0, note:
'10, если билет на вторую и следующую постановку покупки']
  bulk_discount_pct decimal(5,2) [not null, default: 0, note: '10, если в
покупке больше 10 билетов на эту постановку']
  paid_amount decimal(10,2) [not null, note: 'Фактически заплачено за
билет']
}

Table ticket_returns {
  id int [pk, increment]
  ticket_id int [not null, unique, ref: - tickets.id]
  returned_at timestamp [not null]
  retained_amount decimal(10,2) [not null, default: 0, note: 'Удержание 50%
при возврате накануне показа']
  recalculation_amount decimal(10,2) [not null, default: 0, note:
'Удержание за отмену оптовой скидки с оставшихся билетов']
  refund_amount decimal(10,2) [not null, note: 'Выплачено покупателю']
}
```

A.3 Qwen, исправленная схема (P1)

```
//
================================================================================
// ДОПУЩЕНИЯ И ПОЯСНЕНИЯ К СХЕМЕ (ИСПРАВЛЕНИЯ ПО ЗАМЕЧАНИЯМ):
// 1. РАЗДЕЛЕНИЕ СКИДОК: Скидки хранятся раздельно (bulk_discount_rate и
//    multi_show_discount_rate) на уровне OrderProduction, что позволяет
//    однозначно определить причину скидки и корректно пересчитать её при
возврате.
// 2. ОПТОВАЯ СКИДКА НА ПОСТАНОВКУ: Сущность OrderProduction группирует все
//    билеты одной постановки в рамках одного заказа. Поле total_quantity
//    содержит суммарное количество билетов (даже если они разных
категорий),
//    что позволяет корректно применить порог "более 10 билетов".
// 3. ЕДИНЫЙ ИСТОЧНИК ИСТИНЫ: Факт продажи/возврата хранится ТОЛЬКО в
таблице
//    Ticket. Поля returned_quantity в OrderItem и status в Order удалены,
```

```
//    чтобы избежать рассинхронизации. Отчеты строятся агрегацией по
Ticket.
// 4. ФИКСАЦИЯ ПЕРЕРАСЧЕТА: В Ticket хранятся paid_amount (фактически
заплачено),
//    recalculation_clawback (удержание из-за потери оптовой/мульти-скидки
//    оставшимися билетами) и cancellation_penalty (50% за возврат
накануне).
//    refund_amount = paid_amount - recalculation_clawback -
cancellation_penalty.
// 5. ЦЕЛОСТНОСТЬ ЗАЛОВ И КАТЕГОРИЙ: Иерархия Theater -> TheaterHall ->
//    SeatingCategory строго соблюдена. Добавлены бизнес-ограничения (см.
//    комментарии UNIQUE CONSTRAINT и BUSINESS LOGIC CONSTRAINT), требующие
//    от приложения проверять принадлежность категории к залу показа.
// 6. УНИКАЛЬНОСТЬ: Добавлены явные комментарии для составных уникальных
//    индексов (ProductionPrice и Performance), предотвращающих
дублирование
//    цен и наложение показов в одном зале.
// 7. ИСТОРИЧНОСТЬ ВМЕСТИМОСТИ: Вместимость вынесена в
SeatingCategoryCapacity
//    с полями valid_from и valid_to, что позволяет корректно считать
коэффициент
//    "успешности" за прошлые периоды, игнорируя текущие изменения зала.
//
================================================================================

Table City {
  id int [pk, increment]
  name varchar [not null]
}

Table Theater {
  id int [pk, increment]
  city_id int [not null]
  name varchar [not null]
  address varchar
}

Table TheaterHall {
  id int [pk, increment]
  theater_id int [not null]
  name varchar [not null]
}

Table SeatingCategory {
  id int [pk, increment]
  theater_hall_id int [not null]
  name varchar [not null]
  // Вместимость (seat_count) удалена отсюда и перенесена в
SeatingCategoryCapacity (п.7)
}

Table SeatingCategoryCapacity {
  id int [pk, increment]
  seating_category_id int [not null]
  seat_count int [not null]
  valid_from date [not null]
  valid_to date // NULL означает "действует по настоящее время"
}
```

```
Table Play {
  id int [pk, increment]
  title varchar [not null]
  description text
  duration_minutes int
}

Table Production {
  id int [pk, increment]
  theater_id int [not null]
  play_id int [not null]
  start_date date [not null]
  end_date date [not null]
}

Table Performance {
  id int [pk, increment]
  production_id int [not null]
  theater_hall_id int [not null]
  show_time timestamp [not null]
  // UNIQUE CONSTRAINT: (theater_hall_id, show_time)
  // Гарантирует, что в одном зале не может быть двух показов в одно время
(п.6)
}

Table ProductionPrice {
  id int [pk, increment]
  production_id int [not null]
  seating_category_id int [not null]
  price decimal(10, 2) [not null]
  // UNIQUE CONSTRAINT: (production_id, seating_category_id)
  // Гарантирует одну цену на одну категорию для одной постановки (п.6)
  // BUSINESS LOGIC CONSTRAINT: seating_category_id должен принадлежать
  // TheaterHall, который принадлежит theater_id данной production_id (п.5)
}

Table "Order" {
  id int [pk, increment]
  customer_name varchar [not null]
  customer_phone varchar [not null]
  created_at timestamp [not null, default: `now()`]
  total_amount decimal(10, 2) [not null] // Суммарная стоимость всех
OrderItem
  // Статус заказа удален, так как он является производным от статусов
Ticket (п.3)
}

Table OrderProduction {
  // Группирует билеты по постановкам внутри заказа для корректного
  // расчета оптовой скидки (п.2) и разделения ставок скидок (п.1)
  id int [pk, increment]
  order_id int [not null]
  production_id int [not null]
  total_quantity int [not null] // Сумма quantity всех OrderItem для этой
постановки в заказе
  bulk_discount_rate decimal(4, 2) [not null, default: 0.0] // 0.10 если
total_quantity > 10, иначе 0.0
```

```
  multi_show_discount_rate decimal(4, 2) [not null, default: 0.0] // 0.10
если это 2-я и последующая постановка в заказе, иначе 0.0
}

Table OrderItem {
  id int [pk, increment]
  order_production_id int [not null] // Ссылка на группировку по постановке
  performance_id int [not null]
  seating_category_id int [not null]
  quantity int [not null]
  base_price_per_ticket decimal(10, 2) [not null]
  total_price decimal(10, 2) [not null] // quantity * base_price * (1 -
bulk_discount_rate - multi_show_discount_rate)
  // BUSINESS LOGIC CONSTRAINT: seating_category_id должен принадлежать
  // тому же theater_hall_id, что и performance_id (п.5)
  // Поле returned_quantity удалено (п.3)
}

Table Ticket {
  // ЕДИНСТВЕННЫЙ источник истины о факте продажи и возврата (п.3)
  id int [pk, increment]
  order_item_id int [not null]
  paid_amount decimal(10, 2) [not null] // Фактически заплаченная сумма за
этот конкретный билет (п.4)
  status varchar [not null, default: 'sold'] // 'sold' или 'returned'
  returned_at timestamp // Заполняется только при status = 'returned'

  // Поля для прозрачного финансового учета и перерасчета (п.4)
  cancellation_penalty decimal(10, 2) [default: 0.0] // Удержание 50%, если
returned_at < show_time на 1 день
  recalculation_clawback decimal(10, 2) [default: 0.0] // Сумма, удержанная
из возврата из-за потери скидки оставшимися билетами
  refund_amount decimal(10, 2) // Итоговая сумма возврата: paid_amount -
cancellation_penalty - recalculation_clawback
}

//
============================================================================
// СВЯЗИ (RELATIONSHIPS)
//
============================================================================

Ref: Theater.city_id > City.id
Ref: TheaterHall.theater_id > Theater.id
Ref: SeatingCategory.theater_hall_id > TheaterHall.id
Ref: SeatingCategoryCapacity.seating_category_id > SeatingCategory.id

Ref: Production.theater_id > Theater.id
Ref: Production.play_id > Play.id

Ref: Performance.production_id > Production.id
Ref: Performance.theater_hall_id > TheaterHall.id

Ref: ProductionPrice.production_id > Production.id
Ref: ProductionPrice.seating_category_id > SeatingCategory.id

Ref: OrderProduction.order_id > "Order".id
Ref: OrderProduction.production_id > Production.id
```

```
Ref: OrderItem.order_production_id > OrderProduction.id
Ref: OrderItem.performance_id > Performance.id
Ref: OrderItem.seating_category_id > SeatingCategory.id

Ref: Ticket.order_item_id > OrderItem.id
```

A.4 Claude, исправленная схема (P1)

```
// Театральная касса — исправленная схема (P1)
//
// ДОПУЩЕНИЯ:
// 1. «Спектакль» (plays) — произведение, «постановка» (productions) — спектакль в театре
//    с периодом показа, «показ» (performances) — конкретные дата, время и зал.
// 2. Билет продаётся на показ и категорию мест; номера мест не хранятся.
//    Свободные места = вместимость категории на дату показа − невозвращённые билеты.
// 3. Скидки считаются внутри одной покупки: 10% на билеты второй и следующих постановок,
//    10% на все билеты постановки, если на неё куплено больше 10 билетов. Скидки суммируются.
// 4. Возврат возможен не позднее дня накануне показа; накануне удерживается 50% заплаченного.
// 5. Если после частичного возврата на постановку осталось меньше 10 билетов, оптовая скидка
//    с оставшихся билетов отменяется: для каждого из них пишется корректировка в price_adjustments,
//    сумма корректировок удерживается из выплаты по возврату, который их вызвал.
// 6. Билет считается возвращённым, если для него есть запись в ticket_returns.
// 7. Поля theater_id в seat_categories, production_prices, performances и hall_id в tickets
//    повторяют связь, но нужны для составных внешних ключей: они гарантируют, что категория
//    мест, зал и постановка относятся к одному театру, а категория билета — к залу показа.

Table cities {
  id int [pk, increment]
  name varchar [not null]
}

Table theaters {
  id int [pk, increment]
  city_id int [not null, ref: > cities.id]
  name varchar [not null, note: 'Названия театров могут совпадать']
  address varchar
}

Table halls {
  id int [pk, increment]
  theater_id int [not null, ref: > theaters.id]
  name varchar [not null, note: 'Основная сцена, камерная сцена...']
```

```
  indexes {
    (theater_id, name) [unique]
    (id, theater_id) [unique, note: 'Для составных внешних ключей']
  }
}

Table seat_categories {
  id int [pk, increment]
  hall_id int [not null]
  theater_id int [not null]
  name varchar [not null, note: 'Партер, амфитеатр, балкон...']

  indexes {
    (hall_id, name) [unique]
    (id, hall_id) [unique]
    (id, theater_id) [unique]
  }
}

Table seat_category_capacities {
  id int [pk, increment]
  seat_category_id int [not null, ref: > seat_categories.id]
  valid_from date [not null, note: 'Дата, с которой действует вместимость']
  seat_count int [not null]

  indexes {
    (seat_category_id, valid_from) [unique]
  }
}

Table plays {
  id int [pk, increment]
  title varchar [not null]
  author varchar
}

Table productions {
  id int [pk, increment]
  theater_id int [not null, ref: > theaters.id]
  play_id int [not null, ref: > plays.id]
  start_date date [not null, note: 'Начало периода постановки']
  end_date date [not null, note: 'Окончание периода постановки']

  indexes {
    (id, theater_id) [unique]
  }
}

Table production_prices {
  id int [pk, increment]
  production_id int [not null]
  seat_category_id int [not null]
  theater_id int [not null]
  price decimal(10,2) [not null]

  indexes {
    (production_id, seat_category_id) [unique]
  }
```

```
}

Table performances {
  id int [pk, increment]
  production_id int [not null]
  hall_id int [not null]
  theater_id int [not null]
  starts_at timestamp [not null]

  indexes {
    (hall_id, starts_at) [unique, note: 'Один показ в зале в одно время']
    (id, hall_id) [unique]
  }
}

Table purchases {
  id int [pk, increment]
  purchased_at timestamp [not null]
  buyer_name varchar
  buyer_phone varchar
}

Table tickets {
  id int [pk, increment]
  purchase_id int [not null, ref: > purchases.id]
  performance_id int [not null]
  seat_category_id int [not null]
  hall_id int [not null]
  base_price decimal(10,2) [not null, note: 'Цена по прайсу на момент
покупки']
  multi_production_discount_pct decimal(5,2) [not null, default: 0, note:
'10, если билет на вторую и следующую постановку покупки']
  bulk_discount_pct decimal(5,2) [not null, default: 0, note: '10, если в
покупке больше 10 билетов на эту постановку']
  paid_amount decimal(10,2) [not null, note: 'Заплачено при покупке']
}

Table ticket_returns {
  id int [pk, increment]
  ticket_id int [not null, unique, ref: - tickets.id]
  returned_at timestamp [not null]
  retained_amount decimal(10,2) [not null, default: 0, note: 'Удержание 50%
при возврате накануне показа']
  refund_amount decimal(10,2) [not null, note: 'Выплачено покупателю:
заплачено − удержание − корректировки, вызванные этим возвратом']
}

Table price_adjustments {
  id int [pk, increment]
  ticket_id int [not null, ref: > tickets.id, note: 'Оставшийся билет, цена
которого изменилась']
  caused_by_return_id int [not null, ref: > ticket_returns.id]
  amount decimal(10,2) [not null, note: 'Доплата за отмену оптовой скидки
(удерживается из возврата)']
  created_at timestamp [not null]
}

Ref: seat_categories.(hall_id, theater_id) > halls.(id, theater_id)
```

```
Ref: production_prices.(production_id, theater_id) > productions.(id,
theater_id)
Ref: production_prices.(seat_category_id, theater_id) > seat_categories.
(id, theater_id)
Ref: performances.(production_id, theater_id) > productions.(id,
theater_id)
Ref: performances.(hall_id, theater_id) > halls.(id, theater_id)
Ref: tickets.(performance_id, hall_id) > performances.(id, hall_id)
Ref: tickets.(seat_category_id, hall_id) > seat_categories.(id, hall_id)
```

ПРИЛОЖЕНИЕ Б. ЗАПРОСЫ С ЗАМЕЧАНИЯМИ

Б.1 Qwen

Проверь и исправь предложенную схему по следующим замечаниям:

1. OrderItem.discount_rate — одно число на всю позицию, в нём суммированы обе скидки.
Ситуация: в заказе 12 билетов на постановку Б (вторая постановка в заказе), discount_rate = 0.20. Позже покупатель возвращает 3 билета, на постановку остаётся 9 билетов.
Последствие: по одному числу нельзя понять, какая из скидок применена (при discount_rate = 0.10 это может быть и скидка за вторую постановку, и оптовая), поэтому перерасчёт при частичном возврате невозможно выполнить однозначно.
Что изменить: хранить скидку за вторую и следующие постановки и оптовую скидку отдельно.

2. Оптовая скидка привязана к OrderItem (показ + категория мест), а по условию считается по постановке: «более 10 билетов на одну постановку».
Ситуация: в одном заказе 6 билетов в партер и 6 на балкон на одну постановку — это два OrderItem.
Последствие: ни в одной позиции нет 10 билетов, original_quantity не показывает, что на постановку куплено 12 билетов; при возврате непонятно, с какого количества считать порог.
Что изменить: считать и хранить скидки так, чтобы было видно, к какой постановке и каким билетам заказа они относятся.

3. Один факт о возврате хранится в нескольких местах: Ticket.status, OrderItem.returned_quantity, Order.status, а сумма заказа — в Order.total_amount и OrderItem.total_price.
Ситуация: при возврате билета обновили Ticket.status, но не изменили returned_quantity.
Последствие: отчёт по проданным билетам и коэффициент «пустых» продаж дадут разные числа в зависимости от того, какое поле читать.
Что изменить: хранить факт возврата в одном месте, количество и статусы получать из него.

4. При перерасчёте оптовой скидки изменение цены оставшихся билетов нигде не фиксируется — удержание попадает только в refund_amount возвращённых билетов.
Ситуация: куплено 12 билетов со скидкой, возвращено 3 (скидка с 9 оставшихся отменена), затем возвращают ещё 1 билет.
Последствие: сумма второго возврата будет рассчитана от цены со скидкой, хотя скидка уже отменена, — покупателю вернут больше, чем он заплатил; в финансовом отчёте не видно, за что удержаны деньги.
Что изменить: хранить заплаченную сумму по каждому билету и отдельно фиксировать корректировки цены при перерасчёте; отдельно хранить удержание 50%.

5. ProductionPrice.seating_category_id и OrderItem.seating_category_id ссылаются на

категорию любого зала любого театра; Performance.theater_hall_id может указывать на зал другого театра.
Ситуация: постановка идёт в Большом зале, а цена или проданный билет указаны для категории Малого зала (или зала другого театра).
Последствие: продаётся билет на несуществующие в этом зале места; коэффициент «успешности» считается по чужой вместимости.
Что изменить: на уровне ключей обеспечить, чтобы зал показа принадлежал театру постановки, а категория мест билета — залу показа.

6. Нет уникальности: ProductionPrice (production_id, seating_category_id), Performance (theater_hall_id, show_time).
Ситуация: для одной категории введены две цены; в одном зале на одно время назначены два показа.
Последствие: неясно, по какой цене продавать; одни и те же места продаются дважды.
Что изменить: добавить уникальные индексы.

7. Вместимость категорий хранится только текущая (SeatingCategory.seat_count).
Ситуация: после реконструкции зала число мест в партере изменили, нужен коэффициент «успешности» за прошлый год.
Последствие: прошлые показы будут посчитаны с новой вместимостью, коэффициент исказится.
Что изменить: сохранять вместимость с периодом действия.

Сохрани требования исходного задания. Исправь указанные проблемы
и повторно проверь остальные операции, отчёты, ключи, связи и историю.
Не добавляй сущности без необходимости.
Верни один полный блок DBML для dbdiagram.io, а не отдельные фрагменты правок.

Б.2 Claude

Проверь и исправь предложенную схему по следующим замечаниям:

1. Категории мест привязаны к театру, в каждом театре предполагается один зал.
Ситуация: в театре две сцены (основная и камерная) с разным числом мест; один спектакль идёт на основной сцене, другой — на камерной.
Последствие: вместимость показа нельзя определить, коэффициент «успешности» считается по сумме мест всего театра; на камерную сцену можно продать билеты в несуществующий балкон.
Что изменить: ввести залы, привязать к ним категории мест и показы.

2. production_prices.seat_category_id и tickets.seat_category_id могут ссылаться на категорию другого театра; ничто не связывает категорию билета с залом показа.
Ситуация: при вводе цен или продаже выбрана категория «Партер» другого театра.
Последствие: продан билет на места, которых нет в зале показа; отчёты по театру и коэффициент «успешности» искажаются.
Что изменить: обеспечить ключами, что категория принадлежит тому же театру, что и постановка, и тому же залу, что и показ.

3. Перерасчёт оптовой скидки записывается только в ticket_returns.recalculation_amount возвращённого билета; у оставшихся билетов bulk_discount_pct и paid_amount не меняются.
Ситуация: куплено 12 билетов со скидкой, возвращено 3 (скидка с 9 оставшихся отменена), затем возвращают ещё 1 билет.
Последствие: сумма второго возврата считается от paid_amount со скидкой, хотя скидка уже отменена, — покупатель получает больше, чем заплатил; по билету не видно его фактической цены.
Что изменить: хранить корректировки цены каждого оставшегося билета с указанием возврата, который их вызвал.

4. Нет защиты от двух показов в одном зале в одно время: уникален только (production_id, starts_at).
Ситуация: в одном зале на 19:00 назначены показы двух разных постановок.
Последствие: одни и те же места продаются дважды.
Что изменить: уникальность показа по залу и времени.

5. Вместимость категорий хранится только текущая (seat_categories.seat_count).
Ситуация: после реконструкции изменили число мест, нужен коэффициент «успешности» за прошлый год.
Последствие: прошлые показы посчитаются с новой вместимостью.
Что изменить: сохранять вместимость с периодом действия.

Сохрани требования исходного задания. Исправь указанные проблемы
и повторно проверь остальные операции, отчёты, ключи, связи и историю.
Не добавляй сущности без необходимости.
Верни один полный блок DBML для dbdiagram.io, а не отдельные фрагменты правок.